École Nationale Supérieure d'Informatique et d'Analyse des Systèmes - Rabat -

Second-Year Internship Report

Major: Génie de la Data (GD)

Conception of an advanced database migration solution with intelligent schema evolution management

---

Prepared by:

HAFID Haitam

Supervised by:

Mr. SALIHI Lyazid

Jury Members:

Ms. EL FKIHI Sanaa
Ms. ETTAZI Widad

Acknowledgments :

I wish to express my profound gratitude to all those who contributed to the success of this internship. My heartfelt appreciation goes to the supervisory team at RMIT Engineering, with special recognition to **Mr. Salihi Lyazid**, whose invaluable support, dedication, and guidance played a pivotal role in my professional growth and in the successful completion of this project.

I am thankful to the entire development team at RMIT Engineering for their warm welcome, constructive collaboration, and continuous assistance throughout this enriching journey.

Finally, I extend my sincere appreciation to RMIT Engineering as an institution for granting us this remarkable opportunity to acquire knowledge, develop skills, and gain meaningful professional experience.

I would also like to express my deepest gratitude to my jury members, **Ms. ETTAZI Widad** and **Ms. EL FKIHI Sanaa**, for the time they dedicated to evaluating my work, for their insightful comments, and for their invaluable encouragement throughout this academic endeavor.

Abstract :

This project, GQ Migration, presents a complete framework for migrating relational databases from MySQL to PostgreSQL with minimal downtime. The system integrates schema conversion, bulk data snapshotting, and real-time change data capture (CDC) through Debezium and Kafka, ensuring that both source and target remain synchronized throughout the migration process.

The architecture automates schema conversion, initial data loading, and continuous replication, handling differences in SQL dialects, data types, and constraints between MySQL and PostgreSQL. The conversion engine supports both rule-based and intelligent (LLM-assisted) translation for complex schema definitions.

The implementation emphasizes robustness, fault tolerance, and observability. It provides continuous metrics on data lag, event throughput, and synchronization accuracy, ensuring transparency and reliability across all migration phases. By combining automation with real-time validation, GQ Migration delivers a safe, efficient, and consistent transition from MySQL to PostgreSQL.

Keywords: Database Migration, Schema Conversion, Change Data Capture (CDC), Data Synchronization, Monitoring

List of Abbreviations

Abbreviation	Definition
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
CDC	Change Data Capture
LLM	Large Language Model

Terminology

Source Database: A database that contains data to be migrated to one or more target databases. This serves as the origin point for data transfer operations during migration processes.

Target Database: A database that receives data migrated from one or more source databases. This represents the destination system where migrated data will be stored and accessed post-migration.

Database Migration: A migration of data from source databases to target databases with the goal of turning down the source database systems after the migration completes. The entire dataset, or a subset, is migrated based on organizational requirements and constraints.

Homogeneous Migration: A migration from source databases to target databases where the source and target databases are of the same database management system from the same provider. This type of migration typically involves fewer compatibility issues and simpler transformation processes.

Heterogeneous Migration: A migration from source databases to target databases where the source and target databases are of different database management systems from different providers. This approach requires extensive data transformation and mapping to ensure compatibility between different systems.

Database Migration System: A software system or service that connects to source databases and target databases and performs data migrations from source to target databases. These systems typically provide automated tools for schema conversion, data transformation, and migration monitoring.

Zero Downtime: In the context of data migration, zero downtime means that data is transferred or synchronized from the source to the target system without interrupting ongoing operations — users and applications can continue reading and writing data while the migration happens, ensuring continuous availability of the system

Schema Conversion: The process of translating database structures (tables, indexes, constraints) from the source system's SQL dialect to the target system's syntax and semantics.

Change Data Capture (CDC): A mechanism that tracks and streams real-time changes (INSERT, UPDATE, DELETE) from the source database to keep the target synchronized.

Cutover: The final stage of migration where the target system becomes the primary operational database, and the source is decommissioned or switched to read-only mode.

Contents

List of Figures

General Introduction:

In today's digital landscape, data has become one of the most valuable assets for organizations. Businesses rely heavily on databases to store, manage, and analyze critical information. However, as systems evolve and technology advances, many organizations face the challenge of migrating from one database platform to another in order to improve performance, scalability, cost-effectiveness, or compliance. Database migration is therefore a key process in modern information systems, enabling enterprises to adapt to new environments without compromising on data integrity or availability.

The process of migration goes beyond simply transferring data. It requires careful handling of schema conversion, data transformation, and change data capture (CDC) to ensure consistency between the source and target databases. Moreover, the migration pipeline must be designed to minimize downtime, support real-time synchronization, and provide mechanisms for monitoring and validation.

This project, GQ Migration, addresses these challenges by designing and implementing a gateway capable of managing and automating the migration of relational databases. It integrates mechanisms to capture database changes in real time, transform schemas between heterogeneous systems, and apply transformations consistently in the target database. The solution also incorporates auditing and monitoring features, ensuring transparency and reliability throughout the migration lifecycle.

By combining principles of automation, real-time processing, and system integration, the project aims to provide a flexible and scalable approach to database migration. Ultimately, the solution not only facilitates smooth transitions between systems but also contributes to ensuring business continuity during critical migration phases.

Chapter 1

Organization Overview

1.1 Introduction

This chapter provides a general overview of the host organization, RMIT Engineering, where my internship took place. It will cover the company's structure and main areas of expertise, focusing on sectors relevant to my internship experience.

1.2 Overview of RMIT Engineering

Figure 1.1: RMIT Engineering logo

RMIT Engineering, an IT consulting company based in Bordeaux, was founded in 2023. Specializing in the development of digital products and services, the company relies on a team of experienced consultants to support its clients in their projects.

1.3 Areas of Expertise

RMIT Engineering, founded in 2023 and based in Bordeaux, is a consulting company in information systems and software engineering. It supports its clients in their digital transformation with high-quality consulting and development services. Its areas of expertise include the following:

- **Custom Development**: Design and development of tailored web and mobile applications.
- **Artificial Intelligence**: Integration of advanced AI-based technologies to optimize processes and enable intelligent automation.
- **Dematerialization**: Implementation of solutions for electronic document management (EDM) and automated document processing, including OCR Intelligent technology.
- **Sustainable Solutions**: Design of environmentally friendly and innovative solutions that reduce environmental impact.
- **Database Cloud Innovation**: Development of cutting-edge solutions such as GateQuery (SaaS for databases) and AI-driven database migration.
- **Smart Mobility & Content**: Advanced platforms such as Electra Connect (patented geolocation of charging stations in Morocco) and GptNews (next-gen content curation).

RMIT Engineering has collaborated with major clients such as EDF, Arkéa, Solocal, and Ng-Plus. Its projects showcase strong expertise in cybersecurity and modern software architectures. By placing innovation at the heart of its activities, RMIT Engineering develops state-of-the-art solutions in AI, database management, mobility, and sustainable development. These solutions transform key sectors including energy, transport, health, and content management, while providing productivity gains, enhancing user experience, and reducing environmental impact.

1.4 RMIT Engineering Organizational Chart

The organizational chart of RMIT Engineering is characterized by this following structure:

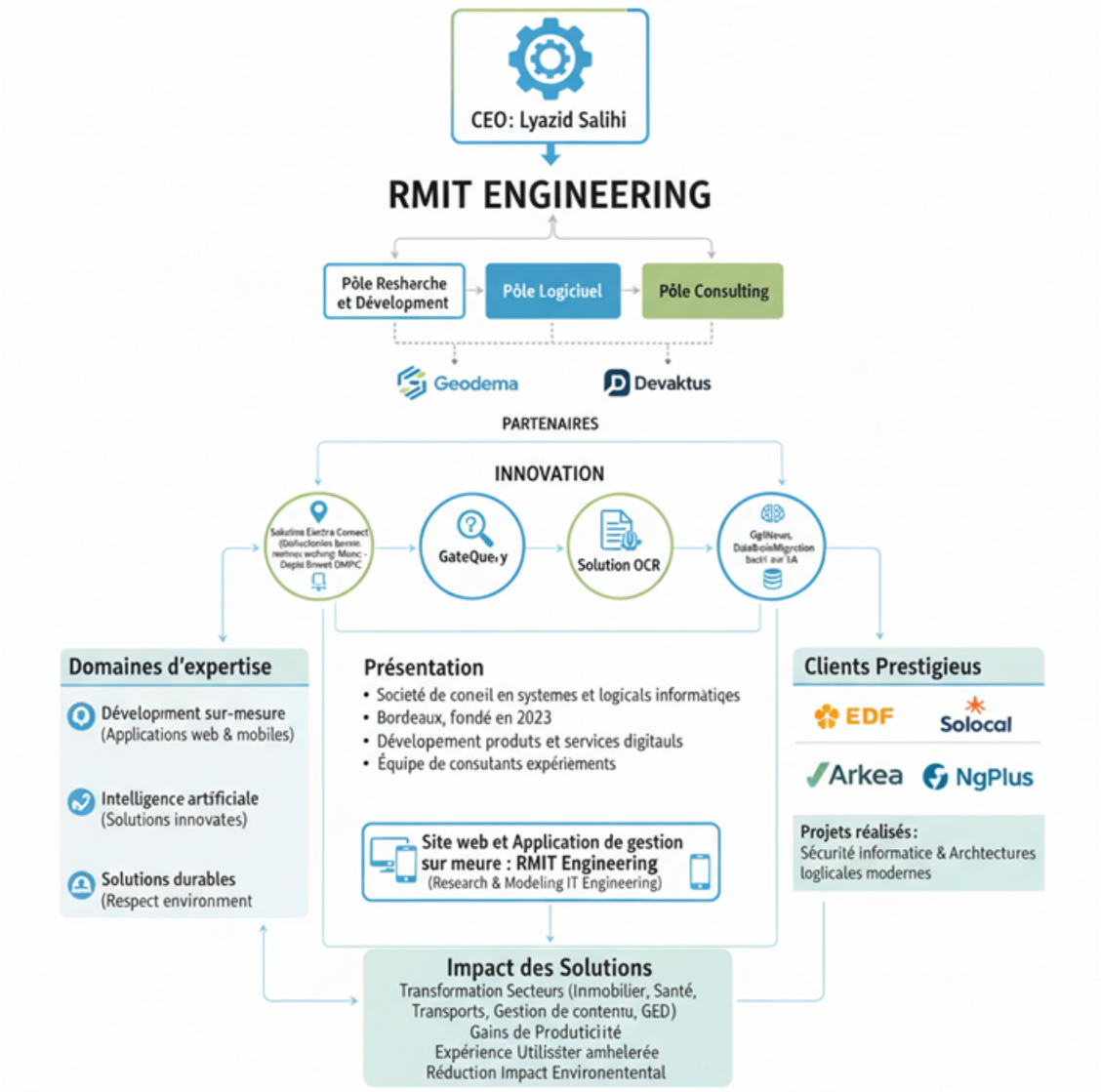

Figure 1.2: Organizational Structure, Domains of Expertise, Innovations, Partners, Clients and Impact of RMIT Engineering

1.5 Conclusion

RMIT Engineering has positioned itself as a dynamic and innovative player in the field of information systems and software engineering. By combining expertise in custom development, artificial intelligence, dematerialization, and sustainable solutions, the company provides its clients with reliable and forward-looking technologies.

The innovative projects developed, such as Electra Connect, GateQue, OCR Intelligent, GptNews, and AI-driven database migration, illustrate its ability to anticipate market needs and design impactful solutions. Its collaborations with major clients such as EDF, Arkéa, Solocal, and NgPlus highlight the trust placed in its know-how and the quality of its services.

By placing innovation and sustainability at the heart of its strategy, RMIT Engineering contributes to the digital transformation of organizations while promoting productivity, enhancing user experience, and reducing environmental impact. Looking ahead, the company is commit-

ted to expanding its partnerships and continuing to deliver cutting-edge solutions that meet the challenges of tomorrow's digital economy.

Chapter 2

General Context of the Project

2.1 Introduction

This chapter provides the foundational context for the heterogeneous database migration system developed during this internship. It explains why this project was needed, what problems it addresses, and what was accomplished.

The chapter is organized into four main sections. First, Section 2.2 explains the motivation behind the project, discussing the business and technical reasons why organizations need better database migration tools. Section 2.3 presents the problem statement, identifying the specific challenges that existing migration solutions fail to address adequately. Section 2.4 outlines the broader project objectives, describing the long-term vision for a comprehensive migration framework. Finally, Section 2.5 defines the specific scope of this internship, detailing what was actually built and delivered as part of the Minimum Viable Product (MVP).

Together, these sections show the progression from identifying real-world problems to developing practical solutions, setting the foundation for the detailed technical discussions in the following chapters.

2.2 Motivation

The motivation behind the developing of this project stems from the technical and business pressure facing modern entreprises. Enterprises today operate in environments where data resides across multiple, and often heterogeneous, database management systems. Migration becomes necessary for a variety of reasons: reducing licensing costs, consolidating infrastructures, adopting open-source alternatives, or ensuring scalability and resilience.

However, database migration is not only about moving data once. Modern organizations expect continuous availability: the source system may remain active during migration, and users demand minimal downtime. This raises the need for mechanisms that can both transfer data efficiently and synchronize subsequent changes in real time.

Furthermore, schema incompatibilities between DBMSs make migrations particularly challenging. A solution that goes beyond simple export–import operations is therefore essential,

one that supports schema transformation, continuous synchronization, and long-term maintainability.

2.3 Problem Statement

Despite the existence of commercial and open-source migration tools, migrations between heterogeneous databases still face several key challenges:

- **Heterogeneous SQL dialects:** Each DBMS has its own syntax and data types, leading to incompatibilities during schema conversion(or translation).
- **Real-time replication:** Capturing and applying changes from the source system without interrupting operations is non-trivial, especially for large-scale datasets.
- **Schema evolution:** When the source schema changes (new columns, altered constraints, etc.), the target must adapt dynamically to preserve data integrity.
- **Monitoring and control:** users need visibility into the pipeline (logs, metrics, dashboards) and mechanisms to intervene when automated processes fail.

This project addresses these issues by proposing a migration framework that not only performs initial full data loads, but also ensures ongoing synchronization of both data and schema between heterogeneous systems.

2.4 Project Objectives

The overarching objective of this project is to design a unified and extensible framework that enables reliable migration between heterogeneous database systems. It seeks to combine initial full data transfer with continuous synchronization mechanisms, ensuring both data and schema remain consistent across platforms. Ultimately, the framework aims to provide organizations with a practical, scalable, and transparent solution that reduces the risks and costs associated with database migration.

2.5 MVP Scope and Deliverables

this internship was specifically tasked with developing and delivering a Minimum Viable Product (MVP) demonstrating core migration capabilities between MySQL (source) and PostgreSQL (target) systems

Within this broader context, during my internship I was tasked to develop and deliver a **Minimum Viable Product (MVP)** that targeted migration between two relational DBMS — **MySQL (source)** and **PostgreSQL (target)** — The internship objectives included:

1. **Database Connectivity**: support for connecting to MySQL and PostgreSQL.

2. **Initial migration (Full Load)**: transfer of all data from selected tables, with automatic creation of the corresponding schema on the target.
3. **Basic CDC**: capture and application of DML operations (INSERT, UPDATE, DELETE) occurring on the source after the initial load using Debezium and Apache Kafka infrastructure.
4. **Assisted schema mapping**: a mechanism for defining simple mapping rules (renaming of tables/columns, basic data type conversion), with a possible extension to ML-based suggestions.
5. **Interface and monitoring**: a lightweight interface (web or CLI) to configure, start/stop, and monitor migration tasks, including error visualization.

2.5.1 Extended Contributions

Upon successfully delivering the MVP, I chose to go beyond the baseline requirements. I extended the CDC mechanism to also handle the **schema evolution** (e.g., column additions, type modifications) on the source during ongoing migration.Additionaly, I enhanced the schena mapping layer by refining existing SQL dialect conversion tools such as SQLGlot . Leveraging regex-based transformations and experimenting with LLM APIs, I built a more intelligent, context-aware translation layer capable of resolving subtle syntactic and semantic differences between MySQL and PostgreSQL — significantly improving migration accuracy and reducing manual intervention.

2.6 System Requirements

The development of the MVP was guided by a set of requirements derived from the problem statement and project objectives:

- **Functional requirements:** The system must connect to both MySQL and PostgreSQL, perform initial full loads, capture ongoing changes via CDC, and apply schema transformations where needed.
- **Non-functional requirements:** The migration process must ensure minimal downtime, fault tolerance in case of failures, and extensibility for future DBMS integration.
- **Interface requirements:** A lightweight interface should allow users to configure, launch, and monitor migration tasks, with visibility into errors and metrics.
- **Operational requirements:** The solution should be deployable in containerized environments and integrate easily with existing open-source components such as Kafka and Debezium.

2.7 Project Planning and Execution

This Gantt chart represents the timeline and key tasks for a project :

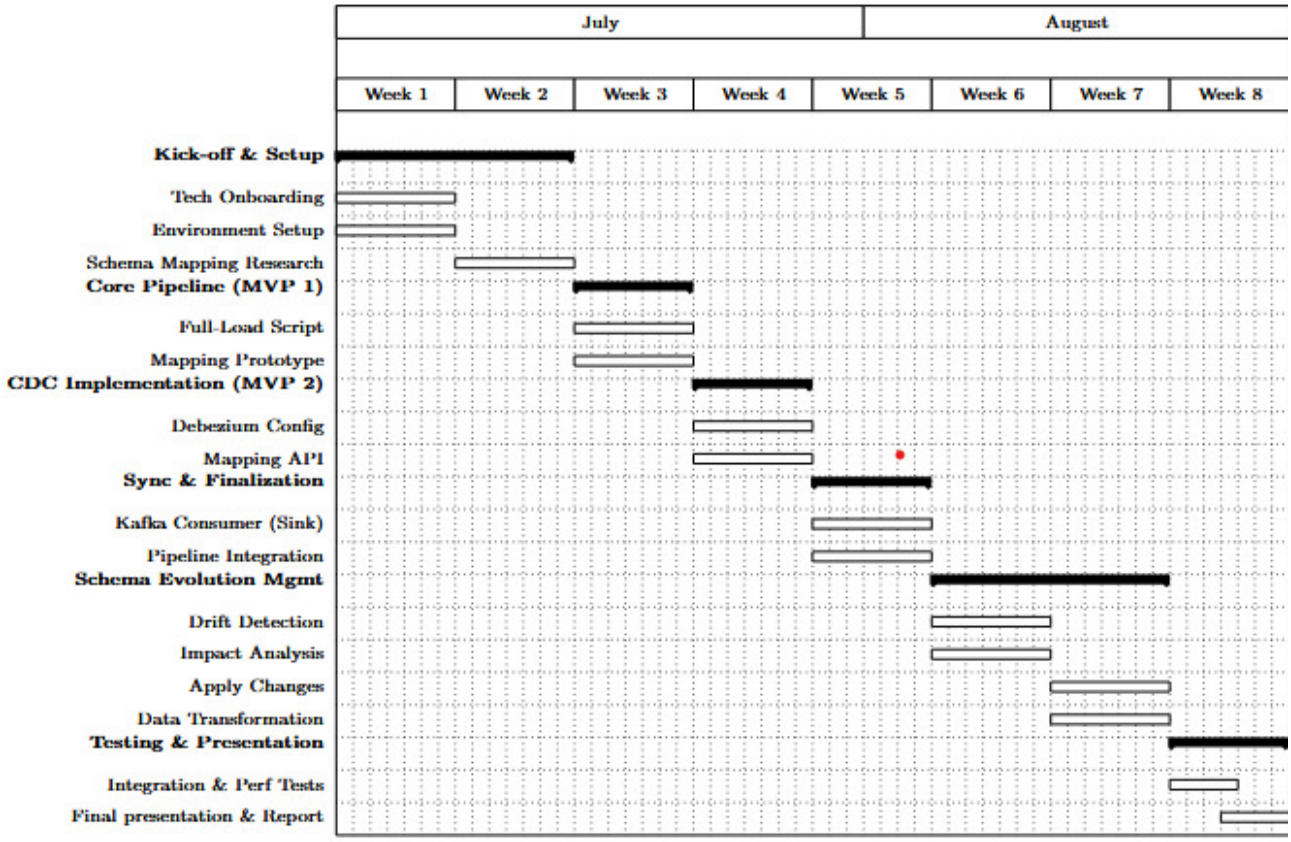

Figure 2.1: Gantt chart

The project followed a well-structured eight-week schedule spanning from July 1st to August 25th, 2025, as illustrated in the Gantt chart above. The first two weeks were dedicated to the Kick-off and Setup phase, focusing on technical onboarding, environment configuration, and initial schema-mapping research. This foundation enabled the smooth transition into the Core Pipeline (MVP 1) phase, where the full-load script and mapping prototype were developed.

During weeks 4 and 5, efforts were directed toward Change Data Capture (CDC) implementation using Debezium and the creation of a Mapping API, ensuring real-time synchronization between MySQL and PostgreSQL. The subsequent Sync and Finalization stage consolidated these components, integrating Kafka consumers and validating end-to-end data flow across the system.

The Schema Evolution Management phase (weeks 6 and 7) emphasized adaptability—detecting schema drift, analyzing potential impacts, and applying data transformation and DDL adjustments. Finally, the last week was devoted to Testing and Presentation, combining integration and performance evaluations with the final demonstration and report submission, marking the culmination of the internship project.

2.8 Conclusion

This chapter established the context and scope for the heterogeneous database migration system developed during this internship. It identified the key motivations driving the need for advanced migration capabilities, including cost reduction, infrastructure modernization, and the requirement for real-time synchronization with minimal downtime. The chapter also outlined the specific technical challenges that existing solutions inadequately address: SQL dialect incompatibilities, real-time replication complexity, schema evolution handling, and limited monitoring capabilities.

The chapter defined both the broader project vision of a comprehensive migration framework and the specific MVP objectives accomplished during this internship. The focus on MySQL-to-PostgreSQL migration, combined with extended contributions in schema evolution and intelligent SQL translation, provides a foundation for understanding the technical solutions presented in subsequent chapters.

Having established the project context and objectives, the next chapter presents a comprehensive literature review of existing database migration approaches and tools. This review analyzes some current commercial and open-source solutions, identifies their specific limitations, and positions the developed system within the broader landscape of migration technologies.

Chapter 3

Literature Review

3.1 Introduction

Database migration has evolved from a simple data transfer task into a complex, strategic operation critical to modern IT systems. Beyond merely copying data, it necessitates ensuring **consistency, integrity, and availability** across heterogeneous systems, often with minimal downtime [5]. This is especially vital in industries with stringent compliance or governance requirements. Traditional approaches, heavily reliant on manual procedures and one-time Extract–Transform–Load (ETL) processes, often resulted in prolonged downtimes, operational risks, and significant resource consumption [1].

With the advent of cloud platforms and distributed systems, database migration has progressed to support **continuous replication and near real-time synchronization**, thereby reducing downtime and enhancing reliability [6]. This chapter explores the core concepts, tools, and frameworks for database migration, tracing the evolution from foundational principles to modern AI-assisted approaches. It concludes by highlighting open challenges and motivating the design choices for the proposed project.

3.2 Conceptual Foundations

3.2.1 From ETL to Continuous Migration

Historically, database migration was dominated by the **Extract–Transform–Load (ETL)** paradigm, a concept central to data warehousing [4]. In this model, data was extracted from the source, transformed into a compatible format, and loaded into the target database in a single, often disruptive, batch. While reliable for static datasets, ETL necessitates significant downtime windows and cannot accommodate ongoing updates in dynamic, high-availability systems.

Modern migration strategies reframe migration as a **continuous process**. An initial full data load is supplemented by **Change Data Capture (CDC)**, which tracks and applies ongoing changes to the target system in near real-time [5, 6]. This paradigm ensures minimal downtime, mitigates data inconsistency risks, and allows organizations to maintain operational

continuity during the migration.

3.2.2 Schema Conversion and Schema Evolution

Schema conversion is the process of translating database structures—including tables, columns, constraints, data types, and indexes—from one database dialect to another (e.g., MySQL to PostgreSQL). It is a critical step, as semantic differences in data types, default behaviors, or constraint handling can lead to data corruption or loss if not meticulously addressed.

Schema evolution refers to managing structural changes to the database schema *after* the migration pipeline is active, such as adding columns or modifying data types. Effective schema evolution is essential for maintaining consistency between source and target systems over time, a particularly acute challenge in continuous migration scenarios [3].

Both schema conversion and evolution are necessary for building a **robust migration pipeline** capable of handling both the initial migration and ongoing changes.

3.2.3 Change Data Capture (CDC)

CDC is the core technology that enables continuous replication. It intercepts changes directly from a database's transaction logs (e.g., binary logs, write-ahead logs) and propagates them to the target system, allowing migrations to proceed without halting the source database [5, 6].

While CDC excels at capturing data manipulation language (DML) events, handling data definition language (DDL) events, or schema changes, remains a significant challenge [3]. Not all CDC tools uniformly detect and propagate schema updates, creating a critical gap in fully automated continuous migration pipelines. Effective CDC, therefore, requires careful coordination between data replication, schema conversion, and monitoring mechanisms.

3.3 Existing Approaches and Frameworks

3.3.1 Managed Cloud Services

Managed services like **AWS Database Migration Service (DMS)** provide streamlined, cloud-native solutions. These services typically perform an initial full load and then continuously replicate changes via CDC. They significantly reduce operational complexity and offer integrated monitoring and alerting [2].

However, managed services are often **restricted to their respective cloud ecosystems** and provide limited support for complex schema evolution or proprietary database features. They also offer less operational transparency, making them less suitable for organizations that require full auditability or on-premise deployment.

3.3.2 Open-Source CDC Frameworks

Open-source tools like **Debezium** and **pgloader** present a flexible, vendor-neutral alternative. Debezium connectors capture transactional changes and stream them to Kafka topics, allowing

downstream consumers to apply these changes to various target systems [5, 6].

This modular, event-driven architecture supports heterogeneous environments and grants full control over the replication logic. The trade-off is a substantial requirement for operational expertise to deploy, monitor, and maintain the pipeline. Furthermore, schema change handling is not uniform across connectors, often necessitating custom logic to maintain consistency [3].

3.3.3 Schema Conversion Approaches

Schema conversion is a critical step in database migration, involving the translation of database structures—including tables, columns, data types, indexes, constraints, and routines—from one system to another. Naïve line-by-line or dictionary-based conversions are often insufficient because differences in SQL dialects, type semantics, and database-specific features can result in incorrect behavior, data loss, or runtime errors [1, 4]. For example, MySQL's `AUTO_INCREMENT`, `ENUM`, and flexible date handling do not directly map to PostgreSQL constructs, requiring careful semantic understanding.

3.3.3.1 Rule-Based Conversion

Traditional tools rely on deterministic, rule-based transformations that apply predefined mappings and syntactic rules [5]. Rule-based approaches are reliable for standard and widely-used constructs, providing predictable outputs and fast processing. However, they often fail for complex or non-standard elements, such as advanced constraints, user-defined types, stored procedures, or cross-table dependencies. Extensive manual intervention is usually required in these cases, which limits scalability and efficiency.

3.3.3.2 AI-Assisted Conversion

Recent research has explored AI-assisted or machine learning methods to infer mappings for ambiguous or complex schema elements [2, 3]. For instance, generative AI models can learn patterns from example databases and propose context-aware conversions for constructs that rule-based systems cannot handle. While promising, pure AI solutions face key challenges:

- **Data scarcity**: Limited high-quality training datasets reduce model reliability.
- **Explainability**: It is difficult to trace why a particular mapping was chosen.
- **Operational risk**: Subtle errors in AI outputs may lead to schema inconsistencies in production.
- **Computational cost**: Training and inference can be resource-intensive, especially for large-scale migrations.

3.3.3.3 Hybrid Rule-Based + AI Approach

A hybrid strategy combines the **strengths of both rule-based and AI-assisted methods**. Standard transformations—such as common type conversions or simple constraint mappings—are handled by deterministic engines (e.g., SQLGlot), ensuring fast, predictable, and

reliable results. Complex or ambiguous cases, such as unusual data types, database-specific functions, or multi-table constraints, are delegated to AI models (e.g., LLMs) capable of semantic reasoning [3, 4].

Beyond correctness, the hybrid approach brings significant **operational and cost advantages**. By relying on deterministic engines for the majority of schema elements, the system avoids excessive API calls to LLMs, keeping request volume low and often within free-tier usage limits. This minimizes external costs and reduces dependency on third-party services. At the same time, we avoid the substantial expense and complexity of training our own models, which would require large datasets, high-performance computing resources, and ongoing maintenance [2, 3].

This hybrid approach offers several advantages:

- **Higher coverage**: AI handles complex cases, ensuring more schema elements are correctly converted automatically, reducing manual effort.
- **Reliability**: Deterministic transformations cover the majority of standard schema elements with predictable results.
- **Efficiency and cost-effectiveness**: Only complex cases invoke AI, lowering computational load, minimizing API calls, and reducing financial cost.
- **Explainability and traceability**: Rule-based steps are fully auditable, while AI outputs can be validated before deployment.
- **Scalability**: Suitable for large, heterogeneous schemas and complex real-world databases without significant overhead.

3.4 Discussion and Identified Gaps

The current landscape of migration solutions reveals a clear fragmentation between the core components: data replication, schema conversion, and schema evolution. Managed cloud services offer convenience at the cost of flexibility and auditability. Open-source frameworks provide extensibility but demand significant expertise and lack comprehensive schema automation. Critically, no single solution provides **complete end-to-end coverage**: seamlessly integrating automated schema conversion, CDC-based replication, robust schema evolution, and granular observability. This identified gap strongly motivates the design of a **modular, on-premise compatible migration framework**. Such a framework would integrate AI-assisted schema conversion, Debezium-based change streams, and built-in observability mechanisms to create a more unified and controllable solution.

3.5 Conclusion

Database migration has fundamentally shifted from static ETL processes to **continuous, low-downtime, and schema-aware operations**. Existing tools, while powerful, address only

isolated parts of the migration lifecycle, leaving significant gaps in the seamless integration of schema conversion, evolution, and operational observability. Open-source technologies such as Debezium [5], Kafka, and pgloader [6] provide excellent foundational blocks for a unified framework.

The project described in this report builds upon these foundations to deliver a **modular, auditable, and CDC-driven migration system**. It aims to bridge the identified gaps, providing a capability to handle heterogeneous database environments with minimal downtime while ensuring consistent data integrity.

Chapter 4

System Implementation and Technologies Used

This chapter details the overall design, core technologies, and practical implementation of the heterogeneous database migration system developed as part of the Minimum Viable Product (MVP). The solution ensures seamless migration and synchronization from MySQL to PostgreSQL through schema conversion, Change Data Capture (CDC), and real-time observability.

4.1 System Architecture

4.1.1 Overview

The system follows a modular, pipeline-based architecture where schema and data changes from the source database are captured, transmitted via a messaging backbone, processed by specialized consumers, and then applied to the target database. A dedicated **Schema Conversion Module** ensures dialect compatibility, while monitoring and human validation are centralized in a Streamlit interface.

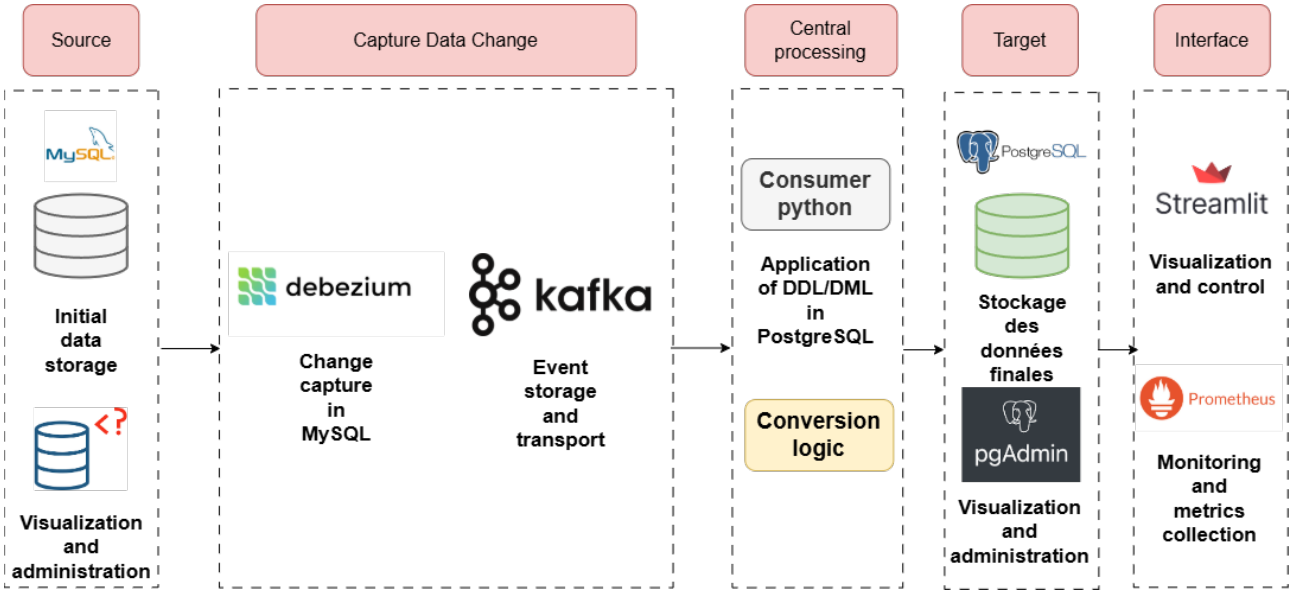

Figure 4.1: High-level architecture of the migration system.

4.1.2 Source and Target Systems

The source is a **MySQL** database containing both schema definitions (DDL) and operational data (DML).

The target is a **PostgreSQL** database, dynamically created from the converted schema and continuously synchronized using CDC.

4.1.3 Change Data Capture Layer

CDC is achieved through **Debezium**, deployed within Kafka Connect, which captures both DDL and DML events from MySQL's binary log. Captured events are serialized and streamed to **Apache Kafka**, ensuring durability, order, and replayability across system restarts.

4.1.4 Messaging Backbone: Apache Kafka

Apache Kafka serves as the communication backbone between the capture and processing layers. Topics are used to separate event types (e.g., DDL, DML, schema history), and offsets ensure at-least-once delivery semantics.

4.1.5 Schema Conversion Module

The **Schema Conversion Module** forms the core of the system. It transforms MySQL syntax into PostgreSQL-compliant syntax through a hybrid process:

- **Rule-based conversion** using `SQLGlot` for deterministic, fast translation.
- **AI-assisted fallback conversion** for complex or MySQL-specific constructs.

SQLGlot parses MySQL statements into Abstract Syntax Trees (ASTs), transforms nodes according to PostgreSQL rules, and regenerates valid SQL. When deterministic rules fail, conversion is delegated to an LLM (via Groq, Gemini, or OpenRouter), with automated validation ensuring syntactic and semantic correctness.

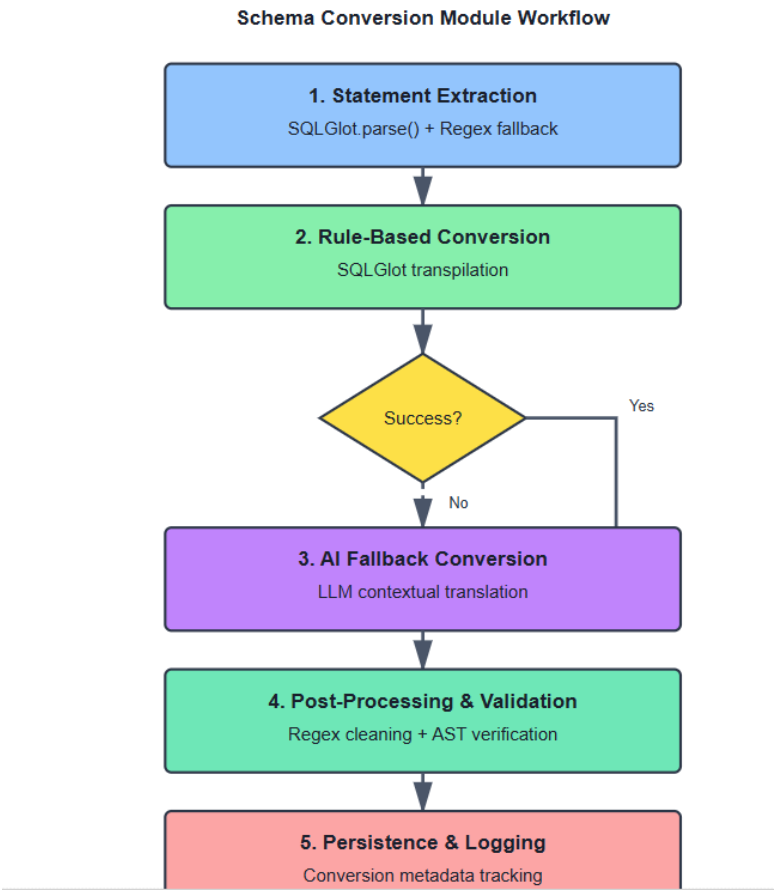

Figure 4.2: Workflow of the Schema Conversion Module.

4.1.6 Monitoring and Observability

Real-time observability is implemented through the **Prometheus Python client**. Metrics on conversion latency, replication lag, and human validation are exposed on port `8001` and visualized directly within Streamlit. Key metrics include:

- `ddl_statements_total`, `dml_statements_total` — processed statement counts.
- `conversion_latency_seconds`, `apply_latency_seconds` — performance indicators.
- `validated_statements_total`, `rejected_statements_total` — user interaction metrics.
- `replication_lag_seconds` — CDC synchronization delay.

This integrated approach eliminates the need for separate Grafana dashboards and provides unified control and analytics in one environment.

4.2 Technologies Used

4.2.1 Database Systems

- **MySQL:** Relational source system providing schema and data to migrate.
- **PostgreSQL:** Target system known for extensibility, JSONB support, and ACID compliance.

4.2.2 Database Administration Tools

- **Adminer:** Lightweight database management tool for MySQL administration and monitoring.
- **pgAdmin:** Comprehensive administration and development platform for PostgreSQL databases.

4.2.3 Messaging and CDC

- **Apache Kafka:** Distributed messaging platform enabling decoupled, scalable event streaming.
- **Debezium:** Open-source CDC engine reading MySQL binlogs and emitting DDL/DML change events to Kafka topics.

4.2.4 Schema Conversion Stack

- **SQLGlot:** Python SQL parser/transpiler used for deterministic conversion.
- **Large Language Models (LLMs):** Groq, Gemini, OpenAI, and others for fallback schema conversion.

4.2.5 Monitoring and User Interface

- **Prometheus Client Library:** For exposing real-time operational metrics.
- **Streamlit:** Lightweight web application framework used for system visualization, monitoring, and human-in-the-loop validation.

4.2.6 Containerization and Orchestration

- **Docker:** Containerization platform used to package and deploy all system components in isolated, reproducible environments.
- **Docker Compose:** Orchestration tool for defining and running multi-container Docker applications, enabling seamless integration of MySQL, PostgreSQL, and administration tools.

4.3 System Implementation

This section presents a detailed description of the implementation of the migration pipeline from MySQL to PostgreSQL, highlighting the role of each component, the flow of data, and the interactions within the system.

4.3.1 Source Database Preparation

The source database was specifically designed to emulate realistic enterprise scenarios, including multiple schemas, tables, and complex relational constraints such as primary keys, foreign keys, unique indexes, and cascading rules. Initial datasets were loaded to represent a range of edge cases, including enumerated types, auto-increment columns, and interdependent constraints. The database structure and data integrity were verified using administrative tools to ensure consistency before initiating the migration.

4.3.2 Change Data Capture Integration

Change Data Capture (CDC) is implemented through the Debezium MySQL connector, which continuously monitors the database binary logs for both schema modifications and row-level data changes. Each captured event is transformed into a structured JSON message containing metadata such as the affected database, table, type of operation, and relevant timestamps. These messages are then published to Kafka topics, providing a reliable, ordered, and replayable stream of events for downstream consumers.

Figure 4.3: Example Debezium CDC payload structure stored in Kafka topics.

4.3.3 Kafka Consumer

A single Python consumer is responsible for processing all change events captured from the MySQL source database and published to Kafka topics. This consumer handles both schema evolution events (DDL) and row-level data modifications (DML) in a unified workflow.

For DDL events, the consumer extracts the MySQL statements and applies the schema conversion module, which first attempts a deterministic, rule-based transformation using SQLGlot. In cases of complex or vendor-specific constructs, an AI-assisted model is invoked to ensure accurate translation into PostgreSQL-compliant statements. All converted statements are validated and logged before being applied to the target database to maintain consistency and traceability.

For DML events, the consumer reconstructs SQL statements from the JSON payloads, which include operation type, primary key information, and the before/after values of each row. These statements are executed in PostgreSQL with idempotency checks and retry mechanisms to prevent duplication or data loss. Throughout processing, the consumer continuously updates operational metrics such as statement throughput, conversion and application latency, and rejected events, supporting both monitoring and human-in-the-loop intervention when necessary.

4.3.4 Schema Conversion Module

The schema conversion module constitutes the core of the migration pipeline. It processes MySQL DDL statements, either captured from Debezium events or extracted from bulk dump files, and generates PostgreSQL-compatible definitions. Standard constructs are handled through SQLGlot's abstract syntax tree transformation, while more complex statements, including multi-column constraints or stored procedures, are resolved via an AI-assisted fallback mechanism. Post-processing steps ensure the removal of MySQL-specific artifacts and the preservation of constraint integrity, producing PostgreSQL statements that accurately reflect the source structure.

In cases where the conversion process encounters errors or ambiguities that cannot be automatically resolved, the pipeline is temporarily halted to allow human intervention. This design choice prevents the propagation of invalid schema changes, which could compromise the integrity of subsequent statements that depend on the affected schema. Operators can review, correct, or approve the conversion through the Streamlit interface before resuming the flow, ensuring that both schema consistency and data integrity are maintained throughout the migration.

The diagram below summarizes the schema conversion pipeline:

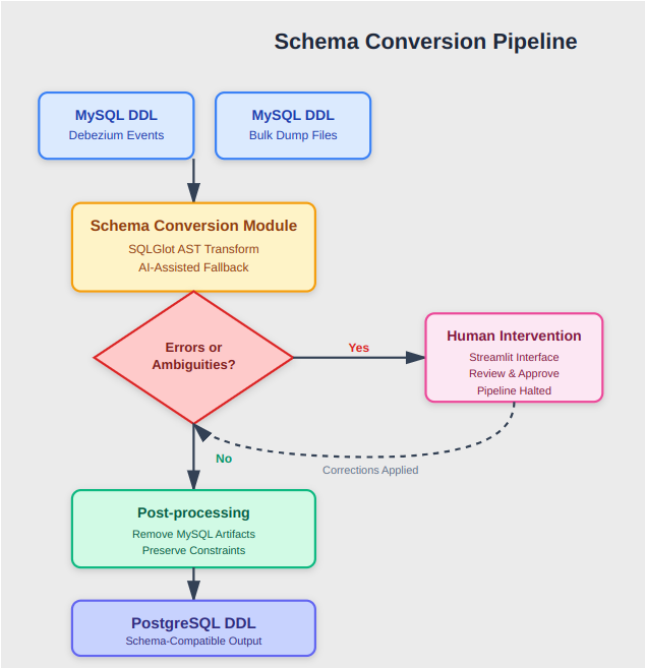

Figure 4.4: Schema conversion pipeline

4.3.5 Full Load Module

The full-load module implements the initial replication phase, establishing a consistent baseline before continuous synchronization begins. The process starts with the extraction of the complete MySQL schema using `mysqldump`, which produces a raw definition file containing all table structures, indexes, and constraints. This file is parsed and processed by the schema conversion module, which translates MySQL DDL statements into PostgreSQL-compatible definitions. Once validated—either automatically or through human review in case of conversion errors—the translated schema is applied to the target PostgreSQL database, ensuring structural equivalence between source and target environments.

After the schema has been deployed, the module proceeds to bulk data transfer. Each MySQL table is exported as a CSV file, preserving column ordering and data types. These files are then loaded into PostgreSQL using optimized batch inserts, ensuring efficient data ingestion while maintaining referential integrity. The process is instrumented with logging and checkpointing mechanisms to allow recovery in case of partial failures. All operations, including schema creation, data import, and anomaly handling, are tracked to ensure completeness and consistency of the initial migration phase.

4.3.6 Streamlit Interface

The Streamlit interface acts as the central control panel of the migration system, offering an integrated view of all operational components. It displays the availability status of core services such as MySQL, PostgreSQL, Kafka, and Debezium, ensuring system readiness before execution.

The interface also provides dedicated sections for key functionalities:

- **Schema Conversion:** allows visualization of MySQL DDL, inspection of the converted PostgreSQL equivalent, and manual correction of statements when automatic conversion fails.
- **Full Load Management:** enables launching and monitoring of initial data transfer operations from CSV exports to PostgreSQL, including progress tracking and anomaly reporting.
- **Monitoring Dashboard:** visualizes Prometheus metrics such as processed statements, validation latency, and replication lag, providing real-time insights into system performance.

Through this unified interface, operators can supervise the entire migration process—schema translation, data loading, and continuous monitoring—without relying on external tools.

4.3.7 Monitoring and Observability

System observability is natively integrated into the Streamlit interface through a Prometheus metrics endpoint exposed by the Python client library. The monitoring layer continuously collects quantitative indicators that describe the operational behavior of the migration pipeline. These metrics are refreshed in real time (every 1s) and visualized within the interface, ensuring that system performance, synchronization lag, and user interactions can be assessed without external dashboards.

The main metrics implemented are as follows:

- `ddl_statements_total` and `dml_statements_total` — counters incremented each time a DDL or DML statement is successfully processed by the consumer. They represent the total volume of schema and data operations, respectively:

$$\text{statements\_processed\_total} = \text{ddl\_statements\_total} + \text{dml\_statements\_total}$$

- `apply_latency_seconds` — measures the time elapsed between event reception from Kafka and its successful application to PostgreSQL. The average application latency is calculated as:

$$L_{\text{apply}} = \frac{\text{apply\_latency\_sum}}{\text{apply\_latency\_count}}$$

- `ddl_success_rate` and `dml_success_rate` — ratios reflecting the percentage of successfully processed statements relative to the total received events:

$$R_{\text{success}} = \frac{\text{successful\_events}}{\text{total\_events}} \times 100$$

- `replication_lag_seconds` — a gauge measuring the temporal delay between the most recent transaction timestamp in MySQL and its application timestamp in PostgreSQL:

$$\text{lag} = T_{\text{source}} - T_{\text{target}}$$

This metric provides a direct indication of the synchronization freshness between the two databases.

Together, these indicators enable comprehensive visibility into the migration pipeline. The integrated visual dashboard aggregates them into intuitive summaries such as success rates, throughput trends, latency distributions, and replication lag over time, allowing operators to quickly detect anomalies and intervene when necessary.

4.4 Conclusion

The implemented system successfully integrates:

- a robust **CDC pipeline** (Debezium + Kafka),
- an intelligent **schema conversion module** (SQLGlot + LLM),
- a real-time **Streamlit interface** for monitoring and human validation,
- and **Prometheus-based observability** for performance tracking.

Together, these components enable both initial migration (full load) and continuous synchronization, validating the feasibility of a hybrid batch + streaming architecture for cross-database migration.

Chapter 5

Results

This chapter presents the results of implementing the MySQL to PostgreSQL migration monitoring and control system. The core achievement is a fully functional, web-based dashboard that provides comprehensive real-time visibility and operational control over the entire migration pipeline. The system successfully integrates monitoring, management, and intervention capabilities into a single, user-friendly interface.

5.1 System Implementation and Validation

The primary result of this work is the successful deployment and integration of the complete migration gateway. The system was validated against the core functional requirements, demonstrating:

- **Real-time Monitoring**: All component statuses and performance metrics update within seconds, providing an accurate live view of the migration health.
- **Operational Control**: Direct management of CDC connectors (pause, resume, restart) is possible without command-line access.
- **Interactive Error Handling**: The system successfully identifies and provides a streamlined interface for correcting failed SQL statements.
- **Automated Readiness Verification**: A rules-based assessment automatically determines if the migration is ready for production cutover.
- **Unified Pipeline Execution**: The entire migration process, from schema creation to CDC initiation, can be executed through a one-click interface.

The following sections detail the key functional areas of the implemented system.

5.2 Comprehensive System Overview

The main dashboard, shown in Figure 5.1, serves as the central hub, providing an at-a-glance status of all critical infrastructure components.

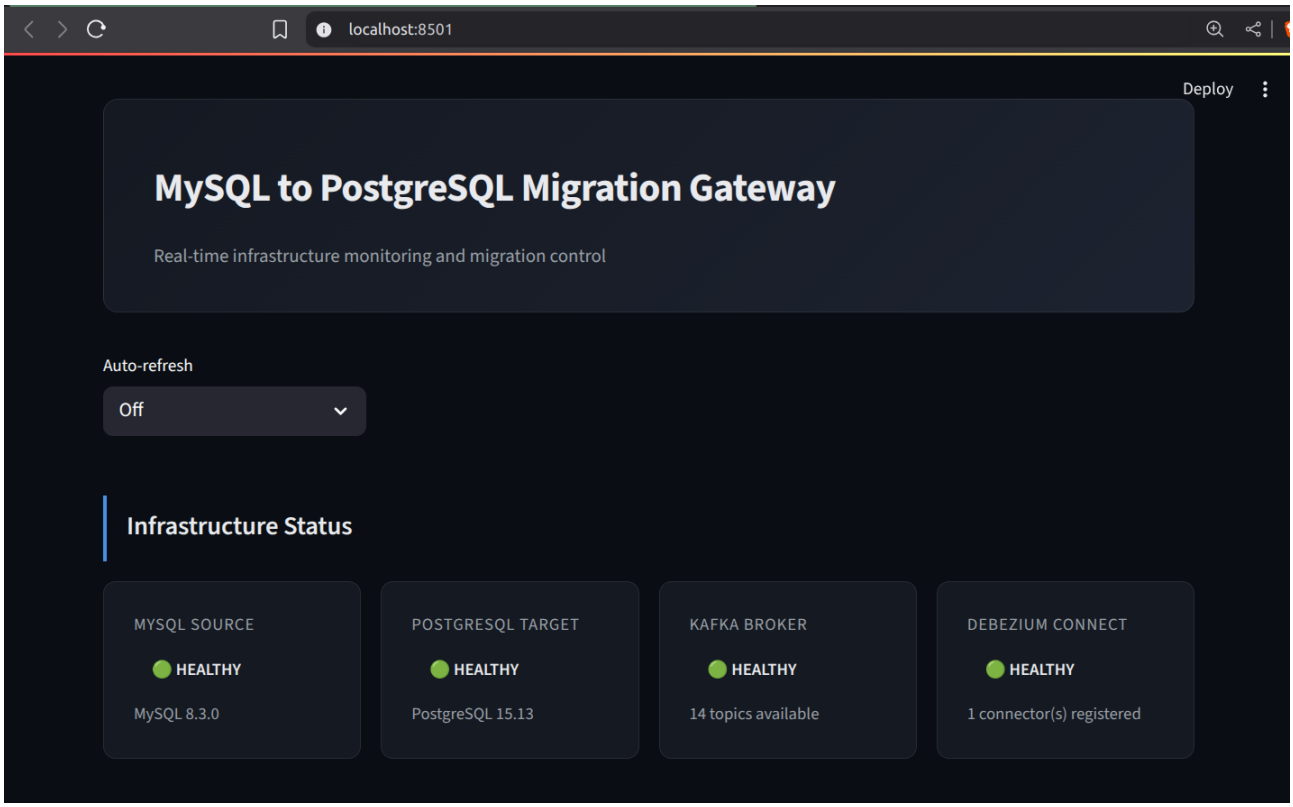

Figure 5.1: Main dashboard showing the operational status of the migration pipeline components.

The interface clearly displays the connectivity and status of:

- **MySQL Source**: Connection status and version information.
- **PostgreSQL Target**: Target database connectivity and responsiveness.
- **Kafka Broker**: Message broker status and active topic count.
- **Debezium Connect**: CDC connector service availability.

5.3 Real-time Performance and Data Metrics

To quantify migration progress and health, the dashboard presents key performance indicators (KPIs) and data inventory.

5.3.1 Migration Performance

As shown in Figure 5.2, the system tracks and displays real-time performance data, which is crucial for assessing the efficiency and stability of the migration process.

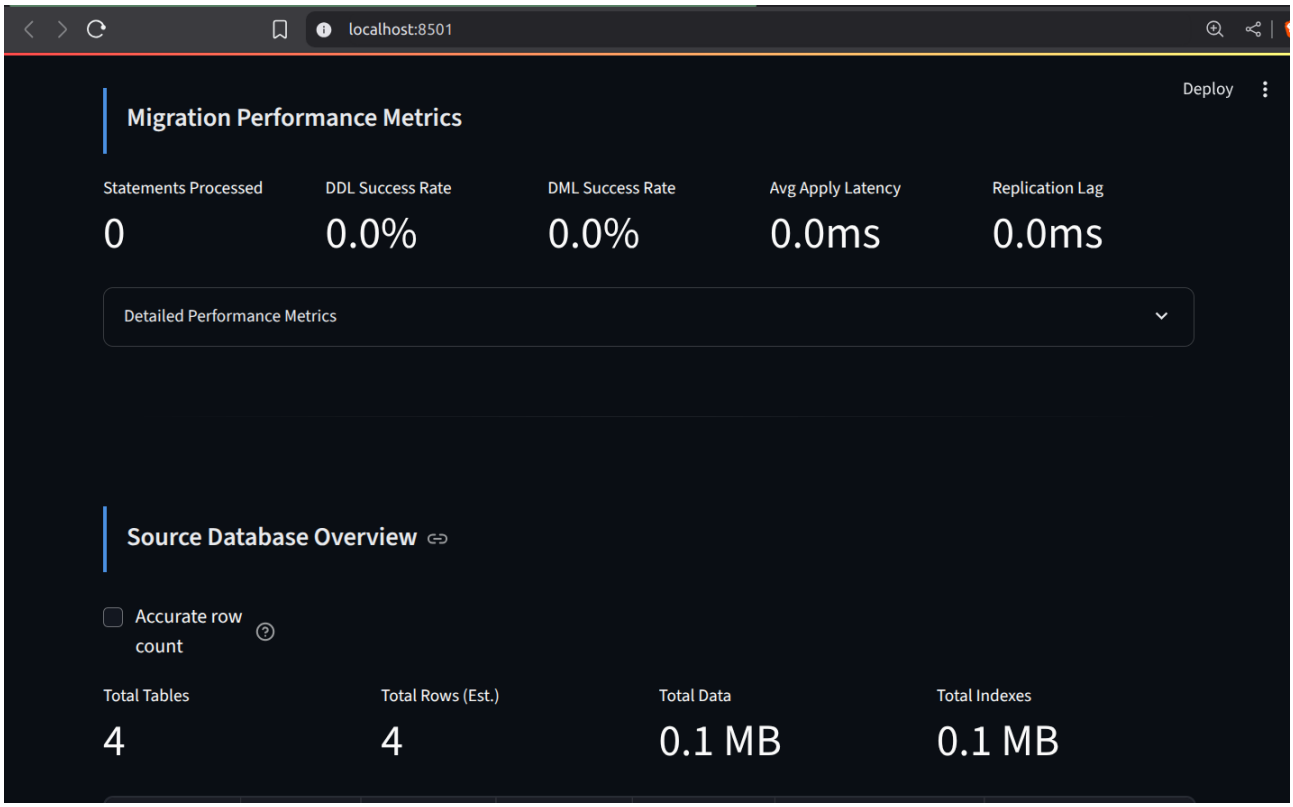

Figure 5.2: Real-time migration performance metrics, including processing statistics and latency.

The metrics panel includes:

- Total statements processed, broken down by DDL and DML.
- Success and error rates for statement application.
- Average apply latency and current replication lag.

5.3.2 Source Database Inventory

Figure 5.3 illustrates the system's ability to provide a detailed inventory of the source MySQL database, offering a clear view of the migration scope.

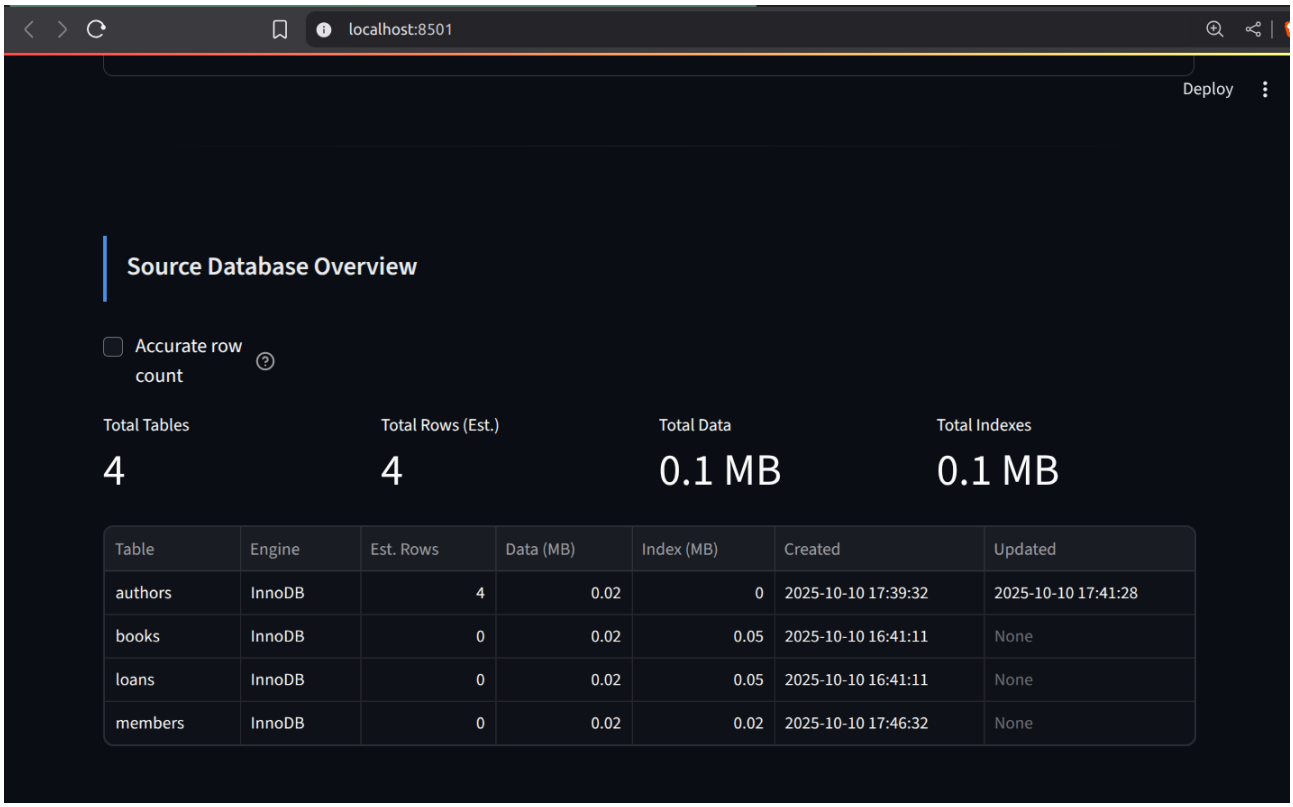

Figure 5.3: Overview of source database tables with size, row counts, and storage statistics.

For each table, the system displays metadata such as:

- Table name, engine, and row count estimates.
- Data and index sizes.
- Creation and update timestamps.

5.4 Operational Control and Management

A key result is the translation of complex backend operations into simple, actionable controls within the dashboard.

5.4.1 CDC Operations Management

The system provides full lifecycle management for Change Data Capture (CDC) components. As seen in Figure 5.4, operators can monitor and control the replication process directly.

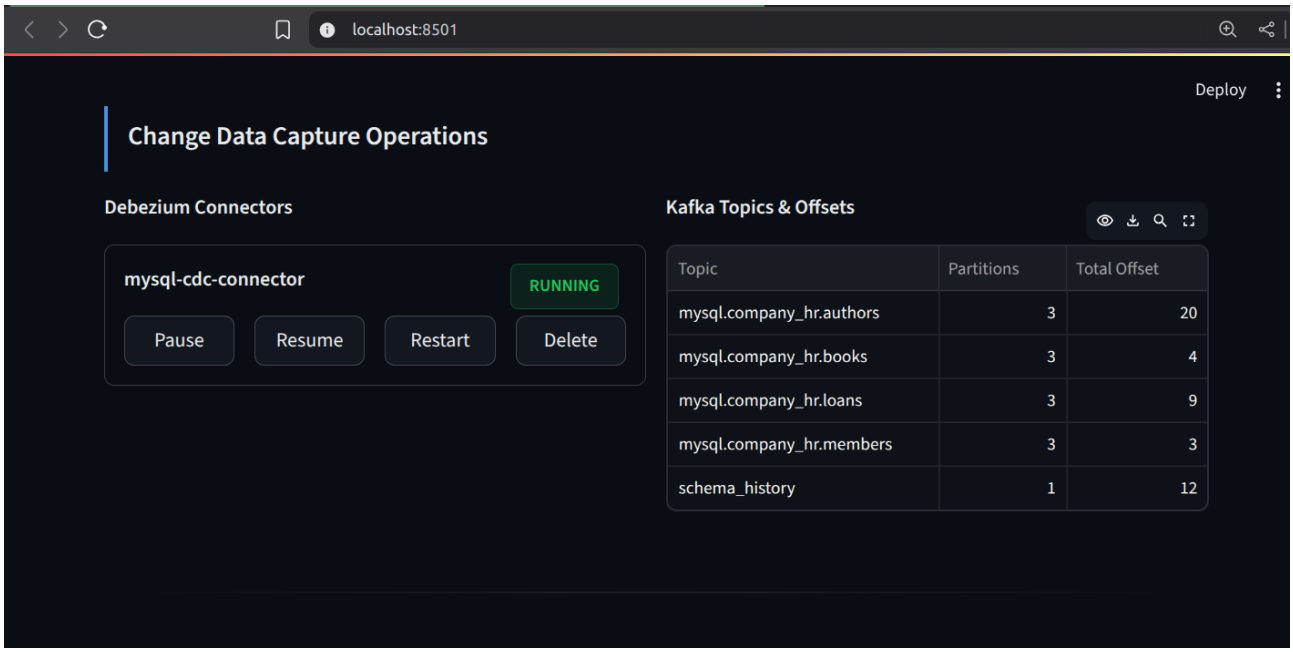

Figure 5.4: CDC operations panel for managing Debezium connectors and monitoring Kafka topics.

This panel provides:

- **Connector Status**: Real-time state (RUNNING, PAUSED, FAILED) of each Debezium connector.
- **Connector Controls**: Buttons to pause, resume, restart, or delete connectors.
- **Kafka Topics**: A list of all CDC topics with their partition counts and current offsets.

5.4.2 Interactive Statement Review and Correction

A critical feature for handling migration failures is the interactive review interface, shown in Figure 5.5. This interface directly addresses the challenge of SQL dialect incompatibilities.

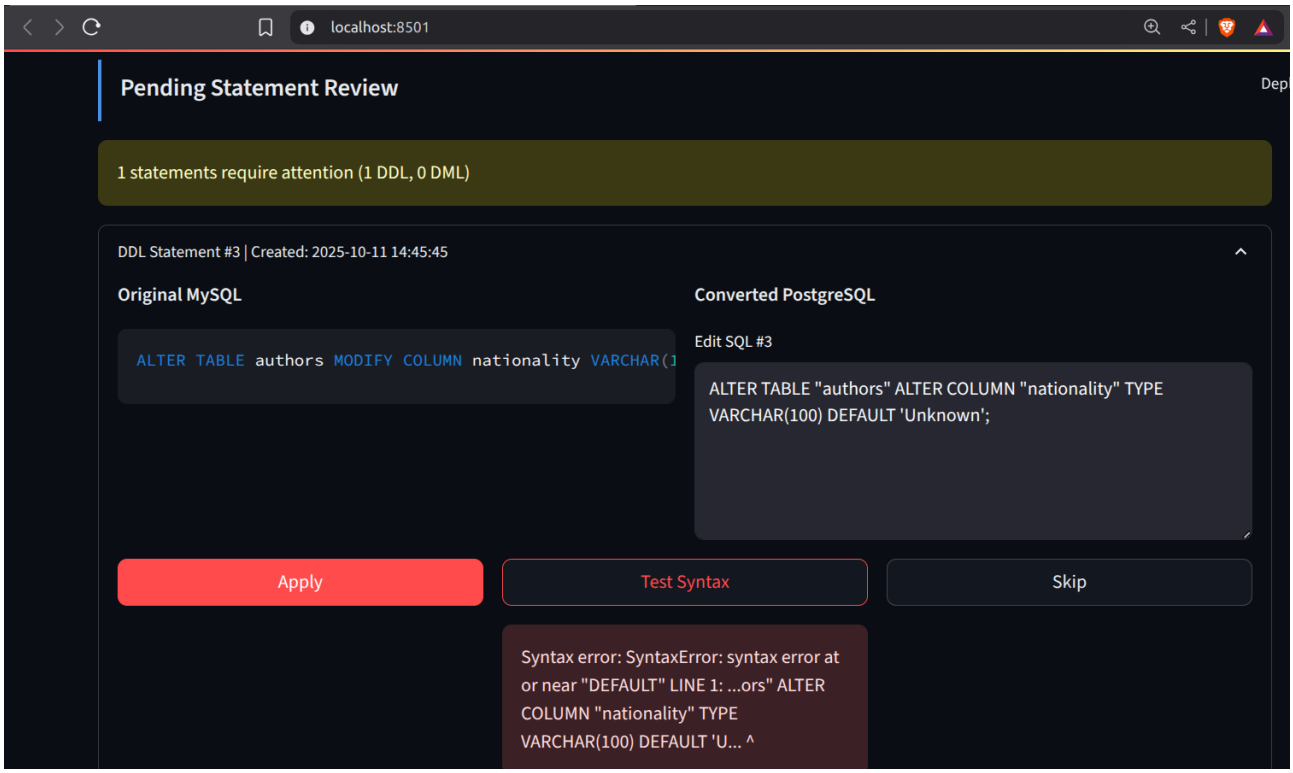

Figure 5.5: Interactive interface for reviewing, editing, and applying pending SQL statements.

For each statement requiring intervention, operators can:

- View the original MySQL statement and the automatically converted PostgreSQL.
- Edit the PostgreSQL SQL directly in the interface.
- Test the syntax of the modified statement before applying it.
- Apply the corrected statement to the target database or skip it entirely.

5.5 Migration Execution and Validation

The system culminates in features that orchestrate the entire migration process and validate its success.

5.5.1 Full Load Pipeline Execution

The interface provides a one-click mechanism to execute the complete initial migration (full load), as depicted in Figure 5.6. This simplifies the often-complex process of bootstrapping a migration.

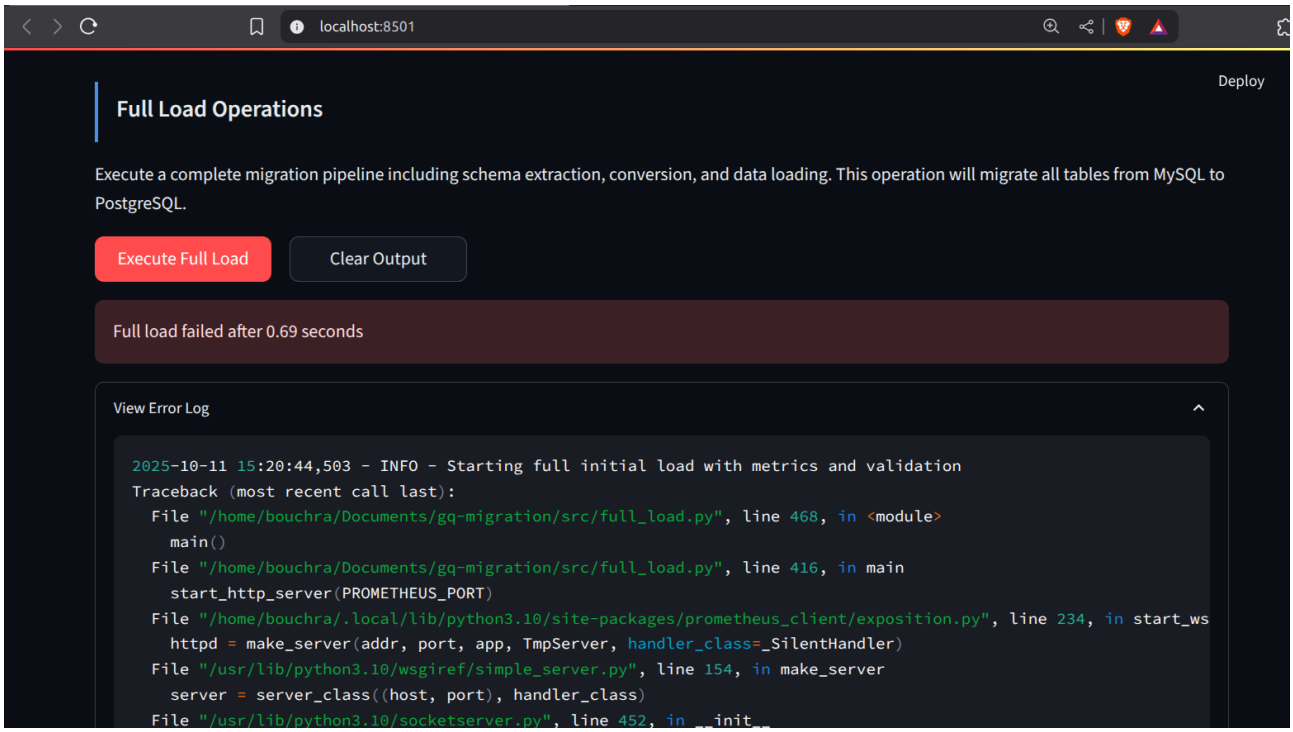

Figure 5.6: Full load pipeline execution interface with integrated log viewer.

This feature enables:

- One-click execution of the end-to-end migration pipeline.
- Real-time visualization of execution progress.
- Access to detailed logs in an expandable viewer for debugging.
- Clear reporting of final status (success/failure) and total execution time.

5.5.2 Automated Readiness Assessment

Before performing a production cutover, the system provides an automated readiness assessment (Figure 5.7), which evaluates critical pre-cutover criteria.

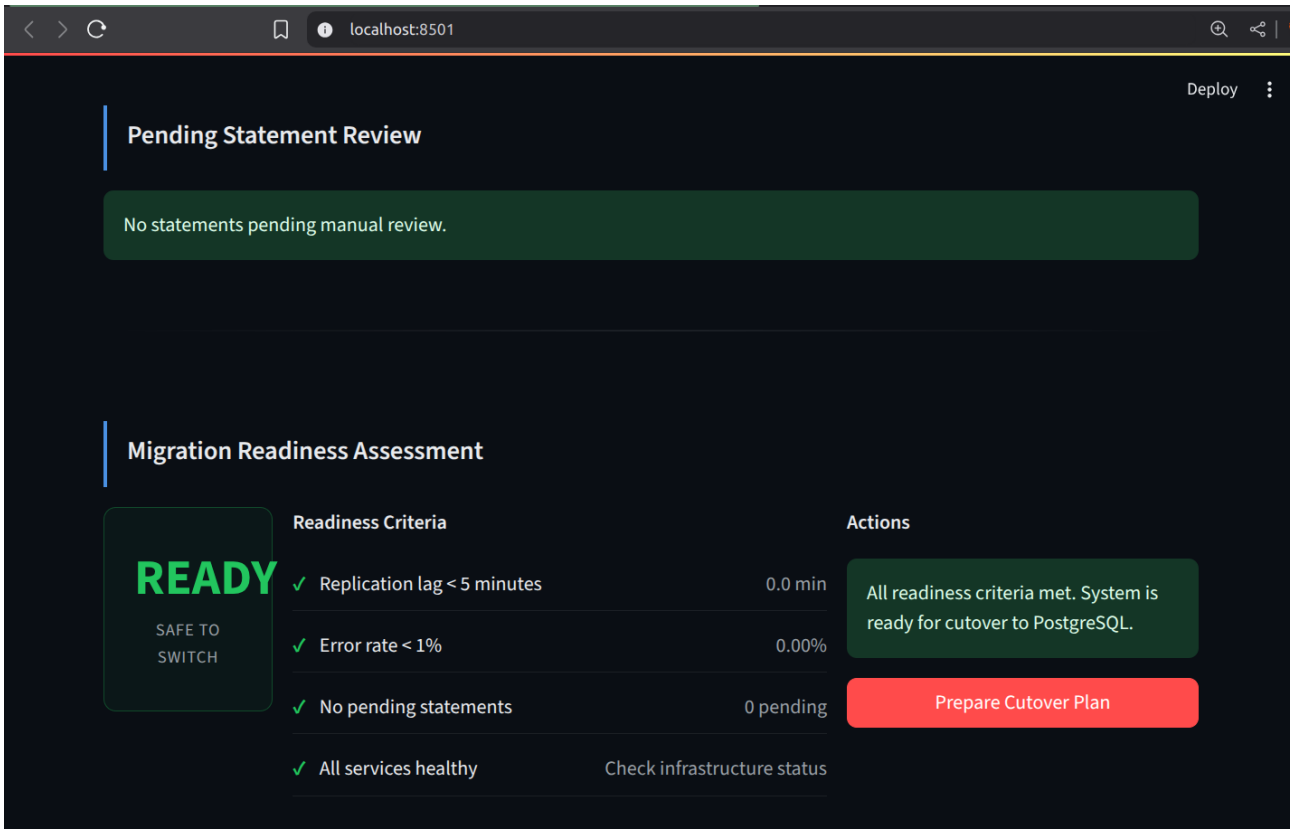

Figure 5.7: Automated migration readiness assessment panel with a criteria checklist.

The assessment checks for:

- Replication lag below a defined threshold (e.g., < 5 minutes).
- An acceptable statement error rate (e.g., $< 1\%$).
- No pending statements awaiting manual review.
- Full health of all underlying infrastructure services.

5.6 Summary

This chapter has demonstrated a successfully implemented and fully functional system for monitoring and controlling MySQL to PostgreSQL migrations. The results confirm that the Streamlit-based dashboard effectively integrates all critical aspects of the migration process:

- It provides **comprehensive, real-time visibility** into the status and performance of the entire pipeline.
- It offers **direct operational control** over CDC components, replacing complex command-line operations.
- It introduces an **efficient, interactive workflow** for handling SQL conversion failures, significantly reducing manual effort.

- It **automates key validation steps**, such as the readiness assessment, to support confident decision-making.

The system thus meets its primary objective: to provide a unified, user-friendly interface that empowers operators to manage a complex database migration with greater efficiency, control, and confidence.

Chapter 6

Assessment and Future Work

6.1 Project Assessment

The project successfully designed and implemented a migration system between two heterogeneous database management systems (DBMS), namely **MySQL** and **PostgreSQL**. The solution combines an initial **full load** with a **Change Data Capture (CDC)** mechanism based on Debezium and Kafka, thus ensuring both the initialization of the target system and its real-time synchronization with the source.

A dedicated **Python consumer** was developed to interpret and apply Debezium events to the target PostgreSQL database, while a **Streamlit interface** was built to centralize operations: service monitoring, schema conversion (DDL), Debezium connector management, full load execution, and source database exploration.

In summary, the system covers the full migration cycle:

- preparation and conversion of the source schema;
- initial migration of existing data;
- continuous replication of changes (CDC);
- supervision and control through a unified interface.

6.2 Identified Limitations

Despite the positive results, some limitations remain:

- schema conversion is still **partially automated**: specific constructs such as `ENUM`, triggers, or stored procedures require manual intervention;
- the solution currently focuses only on the **MySQL–PostgreSQL** ecosystem, without support for other DBMS;
- the Streamlit interface, although functional, lacks advanced monitoring modules such as alerts and graphical dashboards;
- full load performance is highly dependent on data volume and network configuration.

6.3 Future Work

Future improvements can focus on expanding the system's interoperability, automation, and resilience.

- **Multi-DBMS Integration:** Extend the migration pipeline to support additional database engines, such as Oracle, SQL Server, or MongoDB, by designing modular adapters for schema extraction and change event processing.
- **Adaptive Conversion Intelligence:** Integrate fine-tuned LLMs or hybrid rule-based models for more reliable translation of complex MySQL constructs, particularly triggers, functions, and stored procedures.
- **Advanced Workflow Orchestration:** Incorporate orchestration frameworks (e.g., Apache Airflow or Prefect) to manage dependencies between full-load, schema conversion, and CDC modules, allowing automated recovery and retry mechanisms.
- **Performance Optimization:** Enhance full-load throughput through partitioned data ingestion, parallel writes, and compression strategies, reducing overall migration time for large datasets.

6.4 Conclusion

This project successfully demonstrates the implementation of a hybrid migration architecture uniting **batch loading** and **real-time synchronization** within a unified, observable framework. Through the combination of an AI-assisted schema converter, a Debezium–Kafka replication pipeline, and a Streamlit-based control interface, the system achieves accurate schema translation, continuous synchronization, and transparent monitoring. The results validate the design's robustness, showing that heterogeneous database migration can be automated without compromising visibility or control. While further optimization and multi-DBMS support remain open directions, the developed prototype provides a strong foundation for a scalable, fault-tolerant, and production-oriented data migration platform.

General Conclusion :

This project aimed to develop a modular system for automating data migration between heterogeneous database management systems, whether relational or non-relational, while handling schema evolution in real time to ensure consistency between the source and target databases with minimal downtime.

Under the scope of this internship, we successfully implemented a real-time migration pipeline from MySQL to PostgreSQL with support for schema evolution. Specifically, we:

- Developed a schema conversion engine leveraging `sqlglot` for deterministic transformations and an LLM-based fallback for complex SQL DDL statements
- Integrated `Debezium` and `Kafka` to capture change data (CDC) and propagate both DDL and DML operations.
- Created a Streamlit interface for monitoring, reviewing, and approving schema changes before application to the target system with an indicator when it is safe to switch to target database.

A key limitation of the current system lies in the handling of highly complex MySQL-specific constructs (e.g., stored procedures, triggers, and user-defined functions), which still require manual intervention. Additionally, ensuring strict transactional consistency across distributed environments remains an open challenge.

Despite these limitations, the project successfully demonstrates the feasibility of building a robust, modular, and extensible migration framework. It not only addresses immediate migration needs but also lays the foundation for future extensions, such as multi-target replication, integration with non-relational databases, and advanced observability features.

Overall, this work contributes a practical step toward reliable, real-time database migration and evolution, enabling organizations to modernize their infrastructures while minimizing downtime and operational risks.

Bibliography

[1] Shumate, S. E. (2004). *A Study of Database Migration: Understanding the User Experience*. University of Tennessee.

[2] Amazon Web Services. (2024). *AWS Database Migration Service now automates time-consuming schema conversion tasks with generative AI*.

[3] Asokan, E. (2025). *Revolutionizing Database Migration: The Era of AI-Driven Frameworks*. Int. J. of Information Technology and Management Information Systems, 16(2), 162-183.

[4] SQLGlot Documentation. *SQLGlot: Python SQL Parser and Transpiler*.

[5] Debezium. (2023). *Debezium Documentation*.

[6] pgloader. (2022). *pgloader: Loading data into PostgreSQL*.